UAS ↔ ALETHEIA

Combined Deployment and Integration Guide

Version: 1.0 (Engineer Hand-Off)

---

1. Purpose of This Guide

This document describes **how to deploy UAS and ALETHEIA together** as a single, coherent authorization + verification pipeline.

It is intended for **engineering teams** integrating both components in production or production-like environments.

If this guide is followed: - UAS will act as the sole authorization boundary - ALETHEIA will act as the sole verification layer - Deterministic authorization and replayable verification guarantees will hold

This guide assumes familiarity with the individual UAS and ALETHEIA manuals.

---

2. High-Level System Architecture

2.1 Separation of Responsibilities

Component	Responsibility	State Authority
Proposal Generator	Proposes actions	None
UAS	Authorizes proposals (PASS / FAIL / REFUSE)	Yes (authorization only)
Execution System	Executes authorized actions	External
ALETHEIA	Verifies authorization artifacts	None

At no point does ALETHEIA authorize, modify, or block execution.

---

3. Deployment Order

UAS **must be deployed first**.

Recommended order: 1. Deploy UAS 2. Validate UAS authorization behavior 3. Deploy ALETHEIA 4. Integrate ALETHEIA verification against UAS artifacts

Do not attempt to deploy both simultaneously.

---

4. Shared Environment Requirements

4.1 Python Runtime

- Python 3.10 or 3.11
- Identical Python minor versions across UAS and ALETHEIA hosts (recommended)

4.2 Clock Discipline

- System clocks must use UTC
- Clock drift affects timestamps but not authorization logic
- Timestamps must be preserved exactly for replay

---

5. Directory Layout (Recommended)

```
/system/
  /uas/
    uas_v2_standalone.py
    /constraints/
    /evidence/
    /logs/

  /aletheia/
    alethia_verifier.py
    /inputs/
      /uas_decisions/
    /evidence/
    /logs/
```

- UAS writes authorization artifacts to its `/evidence` directory
- ALETHEIA reads **copies** of those artifacts from `/inputs/uas_decisions`

Do not allow ALETHEIA to read directly from UAS working directories.

---

6. Artifact Handoff Contract

6.1 Artifact Type

UAS emits an **Authorization Decision Artifact** containing: - Proposal hash - Verdict - Constraint IDs - Timestamp - Nonce - Canonical decision hash

This artifact is treated as **immutable**.

6.2 Handoff Rules

- Artifacts must be copied, not moved
- Copy must preserve byte-level integrity
- Transport may be file-based, queue-based, or object storage–based

Any transformation invalidates verification guarantees.

---

7. Integration Flow (Step-by-Step)

7.1 Authorization Phase (UAS)

1. Proposal generated by upstream system
2. Proposal serialized
3. UAS evaluates proposal
4. Authorization decision emitted
5. Decision artifact persisted

7.2 Verification Phase (ALETHEIA)

1. Decision artifact ingested
2. Canonical hash recomputed
3. Structural and determinism checks applied
4. Verification Evidence Object (VEO) emitted
5. Verification evidence persisted

UAS and ALETHEIA never call each other directly.

---

8. Replay Workflow

8.1 Required Materials

To replay an end-to-end decision: - Original proposal - UAS constraint set - UAS decision artifact - ALETHEIA verification evidence

8.2 Replay Order

1. Replay UAS authorization
2. Confirm decision hash match
3. Replay ALETHEIA verification
4. Confirm verification hash match

Failure at either stage indicates corruption or misconfiguration.

9. Failure Modes and Interpretation

Symptom	Meaning	Action
UAS FAIL	No policy match	Review constraints
UAS REFUSE	Ambiguous authority	Fix governance error
ALETHEIA fail	Artifact modified	Investigate storage or transport
Replay mismatch	Missing entropy	Restore original timestamps/nonces

Neither UAS nor ALETHEIA failures imply software defects by default.

---

10. Operational Boundaries

10.1 What This System Guarantees

- Deterministic authorization decisions
- Replayable verification evidence
- Clear separation of authority and verification

10.2 What This System Does Not Guarantee

- Correctness of proposals
- Safety of execution
- Persistence durability
- Availability or uptime

These are operator responsibilities.

---

11. Change Management

- UAS constraint changes are governance events
- UAS code upgrades are authorization boundary changes
- ALETHEIA upgrades are verification boundary changes

All changes should be logged and approved.

---

12. Final Guidance

UAS and ALETHEIA are designed to be simple to integrate precisely because they refuse to do anything beyond their defined roles.

Correct deployment depends on respecting those boundaries.

Deviation from this guide invalidates guarantees.